SENIOR ARCHITECT MASTERCLASS

Fine-Tuning

7 Topics across 7 Sections — Teach · Tips · Pocket Notes · Interview · Code · Output

Learn + Code | Pocket Notes | Interview Prep

PEFT (LoRA, QLoRA, DoRA)

Fine-tune huge models on a single GPU

> Imagine you have a brilliant employee who speaks 50 languages and knows every Wikipedia article. You want to teach them YOUR company's specific way of handling refunds. You do NOT need to re-educate them from scratch (that would take years and millions of dollars). You just need a small training session on YOUR specific topic. PEFT = that small training session. It adapts a giant model to YOUR task without retraining the whole thing.

1. LoRA + QLoRA + DoRA + Adapters + Prefix Tuning

📖 DEEP TEACH

THE PROBLEM:
Imagine you want to customize a Boeing 747. Full customization = take the entire plane apart, rebuild every piece. That costs $300 million and takes 3 years.

But what if you just want to add a first-class cabin? You do not rebuild the wings, engines, or fuselage. You ADD a new section inside the existing plane. Much cheaper, much faster, and the plane still flies perfectly.

That is what PEFT (Parameter-Efficient Fine-Tuning) does. Instead of changing ALL 70 billion weights in a model (full fine-tuning), you FREEZE the model and add tiny trainable 'adapters' alongside it. Only 0.1-1% of parameters are trained. The rest stay frozen.

5 PEFT METHODS (from most popular to niche):

LoRA (Low-Rank Adaptation) -- THE STANDARD:
Imagine the model's brain has millions of connections. LoRA adds a SMALL set of new connections alongside the existing ones. The original connections are frozen. Only the new ones learn.

After training, you can MERGE the new connections into the original ones. Result: the model is the same size as before, but now it knows your task. Zero overhead when running.

QLoRA (Quantized LoRA) -- LORA ON A BUDGET:
Imagine you have a beautiful high-resolution photo (70B model = 140GB). It takes up too much space. You compress it to a smaller file (4-bit = 35GB) but it still looks 97% as good. THEN you add LoRA adapters to the compressed version.

Result: A 70 BILLION parameter model fits on ONE GPU instead of needing 4 GPUs. This is what made fine-tuning accessible to startups and individuals.

DoRA (2024, slight upgrade):
Imagine LoRA adjusts both the 'direction' and 'strength' of connections together. DoRA separates them: adjusts direction with LoRA, and strength separately. Slightly smarter approach. +2-5% better than standard LoRA. Drop-in replacement.

ADAPTERS (older method):
Instead of adding connections alongside existing ones, you insert small processing layers BETWEEN existing layers. Like adding extra checkpoints on a highway. Works, but LoRA is generally preferred now.

PREFIX TUNING (no weight changes at all):
Imagine you cannot modify the employee at all, but you CAN write them a really good instruction manual. Prefix tuning learns the PERFECT set of instruction words to prepend to every input. The model weights never change.

WHICH TO USE:
Default (90% of cases): QLoRA (cheapest, fits on consumer GPUs)
Best quality PEFT: DoRA (slight edge over LoRA)
Cannot modify weights: Prefix Tuning
Have lots of GPUs: Full LoRA (skip the compression)
Production deployment: LoRA then merge into base model

🏗 ARCHITECT TIPS + WAR STORIES

Imagine you are a restaurant owner. You do not retrain your entire kitchen staff to add one new dish. You just teach the recipe to your existing chefs. That is PEFT.

1. QLoRA is your default. It lets you fine-tune a 70B model on a $1000 consumer GPU. Nothing else comes close.
2. LoRA rank: start with r=16. Think of rank like detail level. r=16 = good sketch. r=64 = detailed painting. Higher = more

trainable parameters = better but slower.
3. Target modules: start with q_proj and v_proj (attention layers). Add more if accuracy needs improvement.
4. Learning rate: 2e-4 for LoRA (10x higher than full fine-tuning because you are training fewer parameters).
5. UNSLOTH: a library that makes LoRA 2x faster and uses 60% less memory. Free. Always use it.
6. After training: MERGE adapters into the base model. In production, it looks and runs like a normal model -- no adapter overhead.
7. MULTIPLE ADAPTERS: Keep one base model frozen. Swap different LoRA adapters for different tasks. Like having one phone with different apps.

WAR STORY: Startup fine-tuned Llama-70B with full fine-tuning: 8 A100 GPUs ($25K/month), one training run. Switched to QLoRA on 2 consumer RTX 4090 GPUs ($2K total purchase). Same benchmark scores. Training cost: $50 in electricity vs $12K.

📋 POCKET NOTES

LoRA: add small trainable connections alongside frozen model. THE STANDARD.
  Trainable: 0.1-1% of parameters. Quality: 95-99% of full fine-tune.
  Rank r=16 default. Merge after training = zero overhead.
QLoRA: compress model to 4-bit THEN add LoRA. DEFAULT CHOICE.
  70B model: 140GB -> 35GB. Fits on 1 GPU!
DoRA: smarter LoRA (separate direction/strength). +2-5%. 2024.
Adapters: insert layers between blocks. Older. LoRA preferred.
Prefix Tuning: learn perfect instruction words. No weight changes.
Unsloth: 2x faster LoRA, 60% less memory. Always use.
LR: LoRA 2e-4 | QLoRA 1e-4 | Full 2e-5
Multiple adapters: one base model, swap per task (like phone apps)

🎯 INTERVIEW PREP

SAY THIS: 'QLoRA is my default. It compresses the base model to 4-bit and applies LoRA adapters, letting me fine-tune a 70B model on a single GPU. After training, I merge adapters into the base for zero inference overhead. For slightly better quality, DoRA is a drop-in replacement.'

Q: How do you choose LoRA rank?
A: 'Start at r=16. If the task is complex, go to 32-64. I benchmark both on my eval set and pick the best quality-to-cost ratio. Higher rank = more trainable parameters = better capacity but slower training.'

Q: When would you NOT use PEFT?
A: 'When I need to change the model fundamentally -- like teaching it an entirely new language or domain from scratch. That needs full weight updates via continued pre-training. Also for very small models (<1B) where PEFT overhead is not worth it.'

Q: How do you serve multiple LoRA adapters?
A: 'One base model in memory, swap LoRA adapters per request. vLLM supports this natively. One 70B model + 50 LoRA adapters = 50 specialized models at the memory cost of 1.'

⌨ CODE

Python

```python
# QLoRA fine-tuning with Unsloth (2x faster, 60% less memory)
from unsloth import FastLanguageModel

# Load model compressed to 4-bit (QLoRA)
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/llama-3-8b-bnb-4bit",
    max_seq_length=2048,
    load_in_4bit=True,  # Compress to 4-bit
)

# Add LoRA adapters (the small trainable connections)
model = FastLanguageModel.get_peft_model(
    model,
    r=16,                    # Rank 16 = good default
    target_modules=["q_proj","k_proj","v_proj","o_proj"],
    lora_alpha=16,
    lora_dropout=0,
    bias="none",
)

total = model.num_parameters()
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
```

```
print(f"Total parameters: {total:,}")
print(f"Trainable (LoRA): {trainable:,} ({trainable/total*100:.2f}%)")
print(f"Frozen: {total-trainable:,} ({(total-trainable)/total*100:.2f}%)")

# After training: merge adapters into base model
# model.save_pretrained_merged('merged_model', tokenizer)
# Result: normal model file, zero adapter overhead
```

📺 **OUTPUT**

```
Total parameters: 8,030,261,248
Trainable (LoRA): 41,943,040 (0.52%)
Frozen: 7,988,318,208 (99.48%)

# Only 0.52% of the model is being trained!
# The other 99.48% stays exactly as it was.
# Like teaching one new skill without changing anything else.
```

Full Fine-Tuning

When to update ALL weights

> Imagine the difference between teaching your employee a new skill (PEFT) vs sending them back to university for a new degree (full fine-tuning). The second is much more expensive and time-consuming, but sometimes the job requires deep knowledge that a quick training session cannot provide.

2. When, Why, Hardware, Distributed Training

📖 DEEP TEACH

```
FULL FINE-TUNING = update EVERY single weight in the model.

Imagine repainting an entire house vs touching up one room. Full fine-tuning repaints the entire house. It is expensive, time-
consuming, but sometimes necessary.

WHEN YOU NEED THE FULL REPAINT:
1. Teaching the model an entirely new domain. Imagine feeding it 50GB of medical textbooks -- that knowledge needs to soak into
every layer.
2. When PEFT gets you to 90% but you NEED 93% and the 3% gap matters (medical, legal, safety-critical).
3. Small models (under 1 billion parameters) where PEFT overhead is not worth it.

WHEN A TOUCH-UP (PEFT) IS ENOUGH:
1. Adapting to a specific task (classification, extraction, formatting) -- 90% of use cases.
2. Limited budget (you have 1 GPU, not 8).
3. Quality difference is less than 2% -- not worth 10-100x the cost.

HARDWARE REALITY CHECK (how much GPU memory you need):
Think of it like parking trucks. A bigger model = bigger truck = needs more parking spaces.

  7B model:  28 GB (fits in 1 A100 GPU)
  13B model: 52 GB (tight fit in 1 A100)
  70B model: 280 GB (needs 4 A100 GPUs)
  405B model: 1.6 TB (needs 20+ A100 GPUs)

With QLoRA (4-bit compression), divide by ~4:
  70B QLoRA: ~35GB (fits in 1 A100 or 2 consumer GPUs!)

DISTRIBUTED TRAINING (spreading across multiple GPUs):
Imagine a puzzle too big for one table. You split it across 4 tables, each person works on their section, and you assemble the
pieces.

FSDP (PyTorch built-in): spreads the model across GPUs on one machine.
DeepSpeed ZeRO-3: spreads everything (model + training data + calculations) across GPUs. Most memory-efficient.
Axolotl: a framework that handles all this complexity for you.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Imagine you are choosing between a cooking class (PEFT) and culinary school (full fine-tuning). The cooking class teaches you 5 new
recipes in a weekend. Culinary school takes 2 years but makes you a chef. Choose based on what you actually need.

1. ALWAYS try QLoRA first. If the quality gap vs full fine-tuning is less than 2% on your eval set, QLoRA wins (it is 10-100x
cheaper).
2. Cost reality: 7B full fine-tune on cloud GPU = ~$16. 70B full fine-tune = ~$640. 70B QLoRA = ~$8.
3. Mixed precision (bf16): halves memory usage with no quality loss. Always enable it.
4. Save checkpoints every N steps. GPU failures are common on long training runs. Like saving your game -- you do not want to start
over.
5. Gradient accumulation: simulates bigger batches on fewer GPUs. Like cooking in batches when your pot is small.

WAR STORY: Legal AI startup needed deep legal knowledge. QLoRA fine-tuning gave 78% accuracy on legal questions. Full fine-tuning
(continued pre-training on 50GB of case law + instruction tuning) reached 91%. The 13% gap justified the 80x cost because wrong
legal answers have real consequences.
```

📋 POCKET NOTES

Full fine-tune: update ALL parameters. The expensive option.
Analogy: culinary school (2 years) vs cooking class (1 weekend)
Use when: new domain (medical/legal), max quality needed, small models (<1B)
PEFT better: task adaptation, budget limited, quality gap <2%
VRAM: 7B=28GB | 13B=52GB | 70B=280GB | 405B=1.6TB
QLoRA: divide VRAM by ~4. 70B=35GB on 1 GPU.
Distributed: FSDP (single machine) | DeepSpeed ZeRO-3 (multi-machine)
Always: bf16 + gradient checkpointing + save often
Cost: 7B full ~$16 | 70B full ~$640 | 70B QLoRA ~$8
Try QLoRA first. Full only if >2% quality gap.

🎯 INTERVIEW PREP

SAY THIS: 'I default to QLoRA and only escalate to full fine-tuning when benchmarks show a meaningful quality gap -- typically greater than 2%. For a 70B model, QLoRA costs ~$8 vs ~$640 for full. I always try the cheaper option first and let the eval set decide.'

Q: When is full fine-tuning unavoidable?
A: 'Continued pre-training on domain corpora -- feeding the model 10GB of medical literature. PEFT adapters cannot absorb that volume of new knowledge. Also for very small models where PEFT overhead is disproportionate.'

Q: How do you handle multi-GPU training?
A: 'DeepSpeed ZeRO Stage 3 for maximum memory efficiency. Axolotl framework abstracts the complexity. Always enable gradient checkpointing and bf16 mixed precision.'

⌨️ CODE

Python

```python
# Full fine-tuning setup
from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments
from trl import SFTTrainer
from datasets import load_dataset

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3-8B",
    torch_dtype="bfloat16",     # Half memory, same quality
    device_map="auto",          # Spread across available GPUs
)
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3-8B")

# ALL parameters trainable (no PEFT)
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f"Trainable: {trainable:,} (100% -- every single weight)")

dataset = load_dataset("yahma/alpaca-cleaned", split="train[:5000]")

trainer = SFTTrainer(
    model=model, tokenizer=tokenizer,
    train_dataset=dataset,
    dataset_text_field="text",
    args=TrainingArguments(
        output_dir="./full_ft",
        per_device_train_batch_size=2,
        gradient_accumulation_steps=8,  # Small pot, cook in batches
        num_train_epochs=1,
        learning_rate=2e-5,              # LOWER than LoRA (gentler updates)
        bf16=True,
        gradient_checkpointing=True,      # Save memory
        save_steps=100,                   # Save progress often
    ),
)
trainer.train()
```

📺 OUTPUT

```
Trainable: 8,030,261,248 (100% -- every single weight)

# Full 8B model on 1x A100: ~4 hours, ~$16
# Compare QLoRA: only 0.52% of weights, ~30 min, ~$2
# Full = culinary school. QLoRA = cooking class. Choose wisely.
```

Instruction Tuning

Teach the model to follow instructions

> Imagine you download a raw LLM from the internet. It is incredibly smart -- it can complete any sentence. But if you ask 'What is the capital of France?' it might respond 'What is the capital of Germany? What is the capital of Spain?...' because it was trained to CONTINUE text, not ANSWER questions. Instruction tuning teaches it: when a human asks a question, ANSWER it.

3. Alpaca, ShareGPT, ChatML + Data Quality

📖 DEEP TEACH

```
WHAT IS INSTRUCTION TUNING:
Imagine you have a brilliant person who has read every book ever written, but has never had a conversation. They know everything
but do not know HOW to be helpful. You show them 1000 examples of helpful conversations:
  Human: 'Summarize this article.'
  Good response: '3 key points: ...'

  Human: 'Translate to Spanish.'
  Good response: 'Aqui esta la traduccion: ...'

After seeing these examples, they learn the PATTERN of being helpful.

3 DATA FORMATS (how to write the examples):

1. ALPACA FORMAT (like a homework assignment):
  {instruction: 'Summarize this', input: '...long text...', output: '3 key points...'}
  Simple. Good for single-turn tasks. Like filling out a worksheet.

2. SHAREGPT FORMAT (like a chat conversation):
  {conversations: [{from:'human', value:'What is LoRA?'}, {from:'gpt', value:'LoRA is...'},
                    {from:'human', value:'How is it different?'}, {from:'gpt', value:'The difference...'}]}
  Multi-turn conversations. Good for chatbots.

3. CHATML FORMAT (OpenAI's standard):
  <|im_start|>system\
You are helpful.<|im_end|>\
<|im_start|>user\
Hello<|im_end|>
  Includes system prompts. Most production models use this.

THE #1 SECRET: QUALITY > QUANTITY.
Imagine you are training a new doctor. Would you rather they study:
  A) 100,000 patient cases from random internet forums (lots of wrong info)
  B) 1,000 carefully reviewed cases from expert doctors

Obviously B. The Stanford LIMA paper proved this: 1,000 carefully curated examples matched the quality of 52,000 auto-generated
ones.

DATA QUALITY CHECKLIST:
1. DIVERSE: cover many task types, not just one
2. CONSISTENT: all examples follow the same format and style
3. BALANCED: do not let one task type dominate (if 80% is classification, the model forgets how to write)
4. CLEAN: no duplicates, no errors, no contradictions
5. VERIFIED: a human reviewed at least 5% of the data
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of instruction tuning like writing a training manual for new employees. A manual with 1000 clear, diverse examples beats one
with 100,000 sloppy ones.

1. LIMA paper: 1,000 curated examples matched 52,000 auto-generated ones. Quality is EVERYTHING.
2. Start with 1,000 examples. Add more ONLY if your eval metrics improve.
3. DATA POISONING: imagine 200 of your examples say 'I cannot help with that.' The model learns to REFUSE everything. A few bad
```

examples can ruin the whole model.
4. DEDUPLICATION: remove near-identical examples. If 30% of your data is the same pattern repeated, the model overfits to that pattern.
5. AXOLOTL framework: supports all 3 formats, handles tokenization, manages training. Best tool for the job.

WAR STORY: Team generated 50K examples with GPT-4. Model fine-tuned well -- then started refusing 40% of normal questions. Root cause: 200 examples (0.4%) had 'I cannot help with that' as the response. Those 200 poisoned examples taught the model to refuse. Removed them, retrained, problem solved.

📋 POCKET NOTES

3 Formats:
  Alpaca: {instruction, input, output}. Single-turn. Like homework.
  ShareGPT: {conversations:[...]}. Multi-turn chat.
  ChatML: <|im_start|>role\
content. Production standard.
Quality > Quantity: 1,000 curated = 52,000 auto-generated (LIMA paper)
Checklist: diverse, consistent, balanced, clean, human-verified 5%
Start: 1,000 examples. Add more only if eval improves.
Data poisoning: few bad examples ruin the whole model.
Dedup: remove near-identical examples.
Framework: Axolotl (supports all formats)

🎯 INTERVIEW PREP

SAY THIS: 'I follow the LIMA principle -- data quality over quantity. I start with 1,000 carefully curated examples covering diverse tasks. I decontaminate against my eval set, verify a 5% sample, and balance task distribution. Only after confirming quality do I scale up.'

Q: How do you create instruction tuning data?
A: 'Three sources: (1) Domain experts write the best examples (expensive but highest quality). (2) GPT-4 generates synthetic examples with human review (good balance). (3) Converting existing data like logs into instruction format. I always mix sources and human-review a sample.'

Q: How many examples do you need?
A: 'Start with 1,000. Train, evaluate, check where the model fails. Generate more examples targeting failure cases. This targeted approach is 10x more efficient than generating 100K random examples.'

⌨️ CODE

Python

```python
from datasets import Dataset

# ALPACA FORMAT (like a homework worksheet)
alpaca_data = [
    {"instruction": "Classify the sentiment",
     "input": "This product is amazing!",
     "output": "POSITIVE"},
    {"instruction": "Summarize in one sentence",
     "input": "Machine learning is a subset of AI...",
     "output": "ML is an AI subset that learns from data."},
]

# SHAREGPT FORMAT (like a chat conversation)
sharegpt_data = [
    {"conversations": [
        {"from": "human", "value": "What is LoRA?"},
        {"from": "gpt", "value": "LoRA adds small trainable layers alongside frozen model weights."},
        {"from": "human", "value": "How is it different from full fine-tuning?"},
        {"from": "gpt", "value": "Full updates all parameters. LoRA updates only 0.1-1%."}
    ]}
]

# Data quality check
def validate(data):
    issues = []
    for i, ex in enumerate(data):
        if not ex.get("output",""): issues.append(f"Row {i}: empty output")
```

```
        if len(ex.get("output","")) < 5: issues.append(f"Row {i}: output too short")
    print(f"Checked {len(data)} examples. Issues: {len(issues)}")
    for issue in issues: print(f"  {issue}")

validate(alpaca_data)
print(f"\
Formats ready: Alpaca={len(alpaca_data)}, ShareGPT={len(sharegpt_data)}")
print("Next: feed to Axolotl or SFTTrainer for fine-tuning.")
```

📺 **OUTPUT**

```
Checked 2 examples. Issues: 0

Formats ready: Alpaca=2, ShareGPT=1
Next: feed to Axolotl or SFTTrainer for fine-tuning.

# In production: 1,000+ examples, diverse tasks, human-verified.
```

RLHF / DPO / KTO

Teach the model which answers humans prefer

> Instruction tuning teaches the model WHAT to do. Alignment teaches it WHICH response is BETTER. Imagine two restaurant dishes that are both correct -- pasta with red sauce and pasta with white sauce. Alignment is like taste-testing with customers and learning that 70% prefer the red sauce. The model learns to prefer what humans prefer.

4. Reward Modeling, PPO, DPO, KTO

📖 DEEP TEACH

```
THE ALIGNMENT PIPELINE:

STEP 1 -- INSTRUCTION TUNING (already done):
The model can follow instructions. It can cook both dishes.

STEP 2 -- ALIGNMENT (teach it which dish customers prefer):

RLHF (the original, complex method):
Imagine a cooking competition:
1. Chef (LLM) makes 2 versions of a dish.
2. JUDGES (humans) taste both and pick the better one.
3. You train a FOOD CRITIC (reward model) that predicts which dish judges will prefer.
4. The chef keeps cooking, and the food critic guides them toward better dishes.

This is how GPT-4 and Claude were trained. It works great but is COMPLEX -- training a reward model + using reinforcement learning
(PPO algorithm) is tricky and unstable.

DPO (Direct Preference Optimization) -- THE MODERN STANDARD:
Skips the food critic entirely. Instead:
1. Collect pairs: [prompt, GOOD response, BAD response]
2. Train directly: 'make outputs more like the good one, less like the bad one.'
One simple training step. No reward model. No reinforcement learning.
Simpler, more stable, nearly as good. This is what most teams use now.

KTO (the easiest method):
You do not even need PAIRS. Just:
  [prompt, response, thumbs_up] or [prompt, response, thumbs_down]
Like collecting Yelp reviews instead of organizing taste tests.
Easiest data collection but slightly lower quality.

RLAIF (AI judges instead of human judges):
Human judges are expensive ($2K for 10K examples).
Use GPT-4 as the judge instead. 80% as good at 10% of the cost.

WHICH TO USE:
Have preference pairs (A is better than B)? -> DPO
Only have thumbs up/down? -> KTO
Need maximum quality + big budget? -> RLHF
No human judges available? -> RLAIF (GPT-4 judges)
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of it like training a waiter. Instruction tuning taught them to serve food. Alignment teaches them HOW to serve -- with a
smile, at the right pace, anticipating needs.

1. DPO is your default. Simpler than RLHF, similar quality.
2. ALWAYS do instruction tuning FIRST, then alignment. Alignment on a model that cannot follow instructions = polishing a brick. It
is still a brick.
3. Data: 5K-10K preference pairs is a good start for DPO.
4. RLAIF: use GPT-4 to judge which response is better. 80% as good as human judgment at 10% the cost.
5. KTO is amazing when you have implicit feedback (click/no-click, thumbs up/down from production).
6. TRL library: has DPOTrainer and KTOTrainer built in. pip install trl.
```

WAR STORY: Team skipped instruction tuning and went straight to DPO. The model learned to avoid bad responses but could not follow basic instructions. Like teaching a waiter table manners before teaching them what food to bring. Added SFT first, then DPO. Quality: 45% -> 89%.

📋 POCKET NOTES

```
RLHF: train food critic (reward model) + RL (PPO). Complex. GPT-4/Claude method.
DPO: train directly from preference pairs. THE MODERN STANDARD.
  Simple, stable, fast. Needs [prompt, good, bad] data.
KTO: only needs thumbs up/down. Easiest data collection.
RLAIF: GPT-4 judges instead of humans. 80% quality, 10% cost.
Pipeline: SFT first (instruction following) -> THEN alignment
  Never skip SFT. Alignment on bad SFT = polishing a brick.
Data: 5K-10K preference pairs for DPO.
Library: trl (DPOTrainer, KTOTrainer). pip install trl.
Decision: pairs=DPO | thumbs=KTO | max quality=RLHF | no humans=RLAIF
```

🎯 INTERVIEW PREP

SAY THIS: 'Two-stage approach: SFT first for instruction following, then DPO for alignment. DPO is my default because it is simpler and more stable than RLHF with similar quality. For implicit feedback, KTO.'

Q: RLHF vs DPO -- when each?
A: 'DPO for 90% of cases. RLHF when you need an explicit reward model you can inspect and debug -- like safety-critical applications where you want to understand WHY the model prefers certain outputs.'

Q: How do you collect preference data?
A: '(1) Human annotators rank A vs B (expensive, highest quality). (2) RLAIF with GPT-4 as judge (80% quality, 10% cost). (3) Production feedback (thumbs up/down, clicks). I start with RLAIF, validate with human spot-checks.'

⌨ CODE

Python

```python
# DPO Training with TRL
from trl import DPOTrainer, DPOConfig
from transformers import AutoModelForCausalLM, AutoTokenizer
from datasets import Dataset

model = AutoModelForCausalLM.from_pretrained("your-sft-model")
tokenizer = AutoTokenizer.from_pretrained("your-sft-model")
ref_model = AutoModelForCausalLM.from_pretrained("your-sft-model")

# Preference data: for each prompt, a GOOD and BAD response
dpo_data = Dataset.from_dict({
    "prompt": [
        "Explain quantum computing simply.",
        "Write a haiku about programming."
    ],
    "chosen": [  # The response humans PREFERRED
        "Quantum computers use qubits that can be 0 and 1 at the same time, "
        "making certain calculations exponentially faster.",
        "Code flows like water / Bugs hide in the deepest loops / Stack trace reveals all"
    ],
    "rejected": [  # The response humans DID NOT prefer
        "Quantum computing is very complex involving quantum mechanics and "
        "many concepts that are hard to explain simply.",
        "Programming is fun and I like writing code every day."
    ]
})

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    train_dataset=dpo_data,
    tokenizer=tokenizer,
    args=DPOConfig(
        output_dir="./dpo_output",
        learning_rate=5e-7,      # Very gentle (alignment is a nudge, not a shove)
```

```
        beta=0.1,
        num_train_epochs=1,
    ),
)
trainer.train()
print("DPO complete. Model now prefers 'chosen' style responses.")
```

📺 **OUTPUT**

```
DPO complete. Model now prefers 'chosen' style responses.

# The model learned: concise + clear > vague + wordy
# DPO learning rate (5e-7) is much lower than SFT (2e-4)
# Because alignment is a gentle nudge, not a full retraining.
```

Synthetic Data Generation

Use AI to create training data

> Imagine you need 10,000 practice problems for a math textbook, but writing them by hand would take months. Instead, you write 20 really good example problems and ask a smart tutor (GPT-4) to create 10,000 more problems in the same style. That is synthetic data generation -- using a strong AI to create training data for a smaller AI.

5. Self-Instruct + Evol-Instruct + Quality Control

📖 DEEP TEACH

```
TWO KEY TECHNIQUES:

1. SELF-INSTRUCT (quantity from seeds):
Imagine you have 175 hand-written recipe cards. You give them to a chef (GPT-4) and say: 'Write me 50,000 MORE recipes in the same style, but for different dishes.'

The chef reads your style, understands the format, and generates thousands of new recipes. Some are great. Some are mediocre. You filter out the bad ones and keep the good ones.

This is exactly how Stanford created the Alpaca dataset: 175 seed examples -> GPT-3 generated 52,000 instruction examples.

2. EVOL-INSTRUCT (progressive difficulty):
Imagine you have a simple question: 'What is Python?'
You ask GPT-4 to make it HARDER:
  Level 1: 'Explain how Python handles memory management.'
  Level 2: 'Compare Python and Rust memory models with code examples.'
  Level 3: 'Design a memory-efficient data pipeline in Python for 1TB datasets.'

Each evolution makes the question more challenging. Training on progressively harder questions produces smarter models.

QUALITY CONTROL (the most important part):
Generating data is easy. Making sure it is GOOD is hard.

1. LLM-AS-JUDGE: Have GPT-4 score each example 1-10. Keep only 7+.
2. DEDUPLICATION: If 30% of your examples are basically the same thing, the model only learns that one thing. Remove near-duplicates.
3. DIVERSITY CHECK: Make sure you cover many topics, not just one.
4. HUMAN SPOT-CHECK: Randomly review 5% of the data. If you find errors, there are more.
5. GENERATE 2x, KEEP TOP 50%: Throw away the bottom half. Keeps only the best.
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of it like a writing workshop. You cannot just ask 100 people to write stories and use them all. You need an editor (quality control) who reads every story and only publishes the good ones.

1. SEED QUALITY = output quality. If your 175 seeds are sloppy, you get 50,000 sloppy examples.
2. Use GPT-4 (the best model) for generation. The generator's quality = the data's quality.
3. Filter AGGRESSIVELY. Generate 2x what you need. Keep the top 50%.
4. COST: 10K examples via GPT-4 = about $50-100. Compare to $2K+ for human writers. 20-40x cheaper.
5. CONTAMINATION WARNING: make sure your synthetic data does not overlap with your evaluation set. Otherwise your test scores look great but real-world performance is worse.
6. MIX SYNTHETIC + HUMAN: 80% synthetic + 20% human-written gives the best balance of quality and cost.

WAR STORY: Team generated 100K examples. Model benchmarks looked great. Then they discovered 30% were near-duplicates of 5 basic patterns. After deduplication to 40K unique examples, the model was actually BETTER because it learned from diverse data instead of memorizing 5 patterns repeated 6000 times each.
```

📋 POCKET NOTES

```
Self-Instruct: 175 seeds -> GPT-4 generates thousands -> filter for quality
Evol-Instruct: make questions progressively harder (level 1->2->3)
Quality control:
```

```
  LLM-as-judge: score 1-10, keep only 7+
  Dedup: remove near-identical examples
  Diversity: cover many topics, not just one
  Human spot-check: review 5% sample
  Generate 2x, keep top 50%
Cost: 10K via GPT-4 = $50-100 (vs $2K+ human writers)
Seed quality = output quality
Mix: 80% synthetic + 20% human = best balance
```

🎯 INTERVIEW PREP

SAY THIS: 'I use synthetic data generation with strict quality control. Self-Instruct for volume from curated seeds, Evol-Instruct for progressive difficulty. I generate 2x what I need, filter with LLM-as-judge, deduplicate, and human spot-check 5%.'

Q: Is synthetic data as good as human-written?
A: 'For instruction tuning: 80-90% as good when properly filtered. For safety and alignment: no -- you need real human judgments for edge cases. Best approach: synthetic for volume + human for critical examples.'

Q: How to prevent GPT-4 artifacts in synthetic data?
A: 'Post-process to remove verbal tics (Certainly!, As an AI). Vary generation temperature. Use multiple different prompts. Mix synthetic with human examples at 80/20 ratio.'

⌨️ CODE

Python

```python
from openai import OpenAI
import json
client = OpenAI()

# SELF-INSTRUCT: generate from seeds
seeds = [
    {"instruction":"Classify sentiment","input":"Great product!","output":"POSITIVE"},
    {"instruction":"Summarize","input":"ML learns from data...","output":"ML finds patterns in data."},
]

def generate_example(seeds):
    response = client.chat.completions.create(
        model="gpt-4",
        response_format={"type":"json_object"},
        messages=[{"role":"user","content":
            f"Examples: {json.dumps(seeds[:2])}\
\
"
            "Generate ONE new, DIFFERENT example. Different task type. JSON only."}]
    )
    return json.loads(response.choices[0].message.content)

print("SELF-INSTRUCT (generating from seeds):")
for i in range(3):
    ex = generate_example(seeds)
    print(f"  {i+1}. {json.dumps(ex)[:80]}...")

# EVOL-INSTRUCT: make questions harder
def evolve(instruction):
    r = client.chat.completions.create(
        model="gpt-4",
        messages=[{"role":"user","content":
            f"Make this harder and more challenging:\
'{instruction}'\
Return only the evolved question."}]
    )
    return r.choices[0].message.content

print("\
EVOL-INSTRUCT (progressive difficulty):")
q = "What is Python?"
for level in range(3):
```

```
    q = evolve(q)
    print(f"  Level {level+1}: {q[:70]}...")
```

📺 **OUTPUT**

```
SELF-INSTRUCT (generating from seeds):
  1. {"instruction":"Translate to Spanish","input":"Hello world","output":"Ho...
  2. {"instruction":"Extract keywords","input":"Machine learning uses neural...
  3. {"instruction":"Fix grammar","input":"Me and him goes to store","output...

EVOL-INSTRUCT (progressive difficulty):
  Level 1: Explain Python's memory management including garbage collection...
  Level 2: Compare Python and Rust memory models with code examples that...
  Level 3: Design a memory-efficient pipeline in Python for processing 1TB...
```

Continued Pre-Training

Teach the model your entire domain

> Imagine you hire a brilliant generalist doctor. They know everything about general medicine. But your hospital specializes in cardiology. You do not just hand them cardiology patient cases (instruction tuning). You make them READ 10,000 cardiology textbooks and research papers first. After reading all that, they deeply understand the domain. THEN you train them on patient cases. Continued pre-training = making the model read your domain library.

6. Domain Adaptation + When vs Fine-Tuning

📖 DEEP TEACH

```
WHAT IS IT:
You feed the model RAW TEXT from your domain. No questions, no answers -- just text.
The model reads medical papers, legal contracts, or financial reports, page after page.
It learns the vocabulary, patterns, and knowledge of your domain.

HOW IT DIFFERS FROM INSTRUCTION TUNING:
Continued pre-training: reading textbooks (learns KNOWLEDGE)
Instruction tuning: practicing exam questions (learns SKILLS)

Imagine learning to cook French food:
  Continued pre-training = reading 100 French cookbooks cover to cover.
  Instruction tuning = practicing 50 specific French recipes.
  You need BOTH: the knowledge of French cuisine AND the skill to execute recipes.

TYPICAL PIPELINE (in order):
1. Continued pre-training: feed 10GB of domain text (full weight update, expensive)
2. Instruction tuning: feed 5K instruction examples (PEFT/LoRA, cheaper)
3. Alignment (DPO): feed 5K preference pairs (gentle nudge)

Each step builds on the previous. Like: read the textbooks -> practice problems -> get feedback.

WHEN TO USE:
- You have a LARGE domain corpus (at least 1GB of text)
- The base model struggles with domain-specific terminology
- You need DEEP domain understanding, not just task execution

WHEN NOT TO USE:
- Small corpus (<1GB). Not enough data to justify the cost.
- The model already understands your domain well. Try RAG first.
- You just need the model to follow a specific format. Use instruction tuning.

CRITICAL: MIXING
If you feed the model ONLY domain text, it forgets general knowledge. Like studying ONLY cardiology until you forget how to treat a cold.
Solution: mix 80% domain text + 20% general text. The general text prevents 'catastrophic forgetting.'
```

🏗 ARCHITECT TIPS + WAR STORIES

```
Think of it like immersion learning. Living in France for a year (continued pre-training) teaches you French much deeper than taking a French class (instruction tuning). But if you ONLY speak French for a year, you might forget some English (catastrophic forgetting). So mix in some English conversations.

1. VOLUME: at least 1GB of domain text. Less than that = not worth the cost.
2. ALWAYS mix 80% domain + 20% general data. Pure domain = the model forgets how to have normal conversations.
3. Learning rate: lower than SFT (1e-5 to 5e-5). You are gently soaking in knowledge, not aggressively learning tasks.
4. ALWAYS follow CPT with instruction tuning. CPT alone gives knowledge but not skills.
5. COST: this is the most expensive step. 7B model on 10GB of text = about $200-500 on cloud GPUs.

WAR STORY: Legal AI startup did continued pre-training on 50GB of case law. The model went from 40% to 78% on legal questions -- just from reading. Then they instruction-tuned on 5K legal Q&A pairs: jumped to 91%. CPT gave the deep knowledge. Instruction tuning gave the skills to use it.
```

📋 POCKET NOTES

Continued Pre-Training: feed raw domain text. Learns KNOWLEDGE.
Analogy: immersion learning (living in France vs taking a French class)
Pipeline: CPT (read textbooks) -> SFT (practice problems) -> DPO (get feedback)
Mix: 80% domain + 20% general (prevent forgetting)
LR: 1e-5 to 5e-5 (lower than SFT, gentle soaking)
Minimum corpus: 1GB. Less = not worth it.
Cost: 7B on 10GB text = ~$200-500
Always follow CPT with instruction tuning
Eval: domain perplexity should steadily decrease

🎯 INTERVIEW PREP

SAY THIS: 'I use continued pre-training when the base model lacks deep domain knowledge. I always mix 80% domain and 20% general data to prevent catastrophic forgetting, and follow CPT with instruction tuning to teach the model to use its new knowledge.'

Q: CPT vs RAG for domain knowledge?
A: 'RAG retrieves facts at query time -- good for changing information. CPT bakes knowledge into weights -- good for deep understanding of terminology, reasoning patterns, conventions. Best: CPT for foundational knowledge + RAG for current facts.'

Q: How do you know if CPT worked?
A: 'Domain perplexity drops 20-50%. The model uses domain terminology naturally. Benchmark scores improve. I compare: same SFT with vs without CPT.'

⌨ CODE

Python

```python
# Continued Pre-Training setup
from transformers import AutoModelForCausalLM, AutoTokenizer
from transformers import TrainingArguments, Trainer, DataCollatorForLanguageModeling
from datasets import load_dataset

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3-8B",
    torch_dtype="bfloat16"
)
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3-8B")
tokenizer.pad_token = tokenizer.eos_token

# Raw domain text (no instruction format -- just reading material)
# In production: 1-50GB of domain documents
domain_corpus = load_dataset("text", data_files="domain_data.txt", split="train")

def tokenize(examples):
    return tokenizer(examples["text"], truncation=True, max_length=2048)

tokenized = domain_corpus.map(tokenize, batched=True, remove_columns=["text"])
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)

trainer = Trainer(
    model=model,
    train_dataset=tokenized,
    data_collator=data_collator,
    args=TrainingArguments(
        output_dir="./cpt_output",
        learning_rate=2e-5,         # Gentle learning rate
        num_train_epochs=1,
        bf16=True,
        gradient_checkpointing=True,
        save_steps=500,
    ),
)
trainer.train()
print("CPT done. Model now deeply understands your domain.")
print("Next: instruction tune this model on domain-specific Q&A.")
```

📺 **OUTPUT**

```
CPT done. Model now deeply understands your domain.
Next: instruction tune this model on domain-specific Q&A.

# Pipeline: CPT (this) -> SFT (next) -> DPO (final)
# Like: read textbooks -> practice problems -> get teacher feedback
```

Fine-Tuning APIs

Upload data, get a custom model back

> Imagine the difference between cooking at home (self-hosted fine-tuning) vs ordering from a restaurant (API fine-tuning). At home, you control every ingredient but need your own kitchen and equipment. At a restaurant, you tell them what you want and they deliver -- but you cannot see the kitchen or customize the recipe fully.

7. Managed Platforms Compared

📖 DEEP TEACH

```
4 MANAGED PLATFORMS:

OPENAI FINE-TUNING (the restaurant with the best menu):
- Upload a JSONL file with your training examples.
- They fine-tune GPT-4o-mini, GPT-3.5, or GPT-4o on their servers.
- You get a custom model ID back. Use it like any other OpenAI model.
- Cost: ~$25 per million training words. No GPU needed on your end.
- Best for: teams already using OpenAI who want the simplest possible workflow.

TOGETHER AI (the budget-friendly restaurant with open-source options):
- Fine-tune open-source models: Llama, Mistral, etc.
- Cheaper than OpenAI (~$5-10 per million words).
- You can download the resulting model (no lock-in!).
- Best for: teams wanting open models at lower cost.

ANTHROPIC (enterprise catering):
- Custom Claude models for enterprise customers.
- Best for: enterprise teams committed to Claude.

SELF-HOSTED (cooking at home):
- Axolotl + Unsloth on your own GPUs.
- Full control over everything. No data leaves your servers.
- Best for: privacy requirements, full customization, cost optimization at scale.

WHEN TO USE MANAGED vs SELF-HOSTED:
Managed is cheaper when: under 1 million tokens/day (no idle GPU costs).
Self-hosted is cheaper when: over 1 million tokens/day (GPU runs constantly anyway).
Managed is required when: you use proprietary models (GPT-4, Claude).
Self-hosted is required when: data cannot leave your servers (healthcare, finance).
```

🏗 ARCHITECT TIPS + WAR STORIES

Think of it like laundry. For a few shirts, the dry cleaner is easier and cheaper (managed). For a family of 6 doing laundry every day, buying a washer and dryer saves money long-term (self-hosted).

1. OpenAI fine-tuning: start here if you already use OpenAI. 10-minute setup.
2. ALWAYS evaluate on a held-out test set. 'My fine-tuned model is better' needs proof.
3. Most common error: wrong data format. Match the API's expected format EXACTLY.
4. Fine-tune -> evaluate -> add more data for failure cases -> re-tune. 3-5 iterations typical.
5. Together AI: best for open-source models you can download and own.
6. COST TIP: Fine-tune GPT-4o-mini (cheap) instead of GPT-4o (expensive). Often the smaller model + fine-tuning beats the bigger model without it.

📋 POCKET NOTES

```
OpenAI: GPT-4o-mini/3.5/4o. Upload JSONL. ~$25/1M tokens. Simplest.
Together AI: Llama/Mistral. ~$5-10/1M tokens. Download the model.
Anthropic: enterprise only. Custom Claude.
Self-hosted: Axolotl + Unsloth. Full control. Data never leaves.
Decision: simplest=OpenAI | cheap+open=Together | full control=self-host
Managed cheaper: <1M tokens/day. Self-host cheaper: >1M tokens/day.
Managed required: proprietary models. Self-host required: data privacy.
```

Common error: data format mismatch. Match API format exactly.
Iterations: fine-tune->eval->add failure data->retune (3-5 times)

🎯 INTERVIEW PREP

SAY THIS: 'For managed fine-tuning, I choose based on model access and control needs. OpenAI for simplest workflow. Together AI for open-source at lower cost. Self-hosted with Axolotl when I need full control or have privacy requirements.'

Q: OpenAI fine-tuning vs self-hosted?
A: 'OpenAI when: using GPT models, want zero GPU management, data is not sensitive. Self-hosted when: need open models, data privacy, full hyperparameter control, or cost optimization at scale. The crossover point is roughly 1 million tokens per day.'

Q: How do you know fine-tuning worked?
A: 'Held-out test set accuracy improves vs base model. Check for overfitting: training loss drops but eval loss increases. Production A/B test: 10% traffic to fine-tuned model, measure quality.'

⌨️ CODE

Python

```python
# OpenAI Fine-Tuning (simplest managed approach)
from openai import OpenAI
import json
client = OpenAI()

# Step 1: Prepare data (JSONL format)
training_data = [
    {"messages": [
        {"role": "system", "content": "You classify customer feedback."},
        {"role": "user", "content": "The app keeps crashing on Android"},
        {"role": "assistant", "content": '{"category":"bug","platform":"android","severity":"high"}'}
    ]},
    {"messages": [
        {"role": "system", "content": "You classify customer feedback."},
        {"role": "user", "content": "Love the new dark mode!"},
        {"role": "assistant", "content": '{"category":"praise","feature":"dark_mode","severity":"none"}'}
    ]},
]

with open("training.jsonl", "w") as f:
    for item in training_data:
        f.write(json.dumps(item) + "\
")

# Step 2: Upload
file = client.files.create(file=open("training.jsonl","rb"), purpose="fine-tune")
print(f"Uploaded: {file.id}")

# Step 3: Start fine-tuning
job = client.fine_tuning.jobs.create(
    training_file=file.id,
    model="gpt-4o-mini-2024-07-18",
    hyperparameters={"n_epochs": 3}
)
print(f"Job: {job.id} | Status: {job.status}")
print("\
# Runs on OpenAI servers. No GPU needed.")
print("# Takes 10-30 min for small datasets.")
print("# Result: use like any OpenAI model with your custom model ID.")
```

📺 OUTPUT

```
Uploaded: file-abc123
Job: ftjob-xyz789 | Status: validating_files

# Runs on OpenAI servers. No GPU needed.
# Takes 10-30 min for small datasets.
```

```
# Result: use like any OpenAI model with your custom model ID.
# Like ordering a custom dish -- you give the recipe, they cook it.
```

LIBRARIES

transformers	Model loading/training	`pip install transformers`
peft	LoRA, QLoRA, adapters	`pip install peft`
trl	SFT, DPO, KTO trainers	`pip install trl`
datasets	Data loading	`pip install datasets`
axolotl	Training framework	`pip install axolotl`
unsloth	2x faster LoRA	`pip install unsloth`

ARCHITECTURE DIAGRAM

Renders on GitHub — or paste into mermaid.live

```
graph TB
    Need([Need Better LLM Performance]) --> WhatKind{What do you need?}

    WhatKind -->|Different BEHAVIOR<br/>format, tone, style| FT[Fine-Tune]
    WhatKind -->|New KNOWLEDGE<br/>facts, docs, policies| RAG[Use RAG<br/>No training needed]
    WhatKind -->|Deep DOMAIN understanding<br/>medical, legal, finance| Full[Full Pipeline:<br/>CPT → SFT → DPO]
    WhatKind -->|Just needs better output<br/>< 500 examples available| Prompts[Better Prompts First<br/>Not enough data yet]

    FT --> HaveGPU{Have GPU?}

    HaveGPU -->|No| API
    HaveGPU -->|Yes| Local

    subgraph API [API Fine-Tuning — No GPU]
        OpenAI_FT[OpenAI Fine-Tuning<br/>Upload JSONL, ~$25/1M tokens<br/>Fine-tuned mini often beats base 4o]
        Together_FT[Together AI<br/>Llama/Mistral, ~$5-10/1M<br/>Download the model after]
    end

    subgraph Local [Self-Hosted — Full Control]
        QLoRA_Path[QLoRA — THE DEFAULT<br/>0.1% params trained<br/>95-99% quality<br/>70B fits on 1 GPU]
        LoRA_Path[LoRA<br/>0.1% params, more VRAM]
        FullFT[Full Fine-Tune<br/>100% params, 4+ GPUs<br/>Only if QLoRA gap > 2%]
    end

    QLoRA_Path --> Unsloth[Unsloth<br/>2x faster, 60% less memory<br/>Always use]
    LoRA_Path --> Unsloth

    subgraph Data [Data Pipeline]
        direction TB
        Seeds[5 Seed Examples] --> SelfInstruct[Self-Instruct<br/>GPT-4 generates thousands]
        SelfInstruct --> EvolInstruct[Evol-Instruct<br/>Progressive difficulty]
        EvolInstruct --> Judge[LLM-as-Judge<br/>Score 1-10, keep 7+]
        Judge --> Dedup[Dedup + Diversity Check]
        Dedup --> Format{Format}
        Format --> Alpaca[Alpaca<br/>Single-turn]
        Format --> ShareGPT[ShareGPT<br/>Multi-turn chat]
        Format --> ChatML[ChatML<br/>Production standard]
    end

    Unsloth --> SFT[SFT Training<br/>Instruction following]
    FullFT --> SFT
    API --> SFT

    SFT --> Alignment{Need alignment?}

    Alignment -->|Have preference pairs| DPO[DPO<br/>Modern standard<br/>5K-10K pairs]
    Alignment -->|Have thumbs up/down| KTO[KTO<br/>Easiest data]
    Alignment -->|Max quality budget| RLHF[RLHF/PPO<br/>Complex but best]
    Alignment -->|No humans available| RLAIF[RLAIF<br/>GPT-4 judges<br/>80% quality, 10% cost]
    Alignment -->|No| Deploy

    DPO --> Deploy
    KTO --> Deploy
    RLHF --> Deploy
    RLAIF --> Deploy

    subgraph Deploy [Deployment]
        Merge[Merge LoRA Adapters<br/>Zero inference overhead]
        Merge --> Quantize[Quantize to 4-bit<br/>Distill + quantize = 100x cheaper]
        Quantize --> Serve[Serve via vLLM]
    end

    Full --> CPT[Continued Pre-Training<br/>Feed raw domain text<br/>80% domain + 20% general]
    CPT --> SFT

    subgraph Cost [Cost Reality]
        C1[1K synthetic examples: $5-10]
        C2[Manual writing: $2,000+]
        C3[7B QLoRA training: ~$8]
```

```
        C4[70B QLoRA training: ~$8]
        C5[70B full fine-tune: ~$640]
    end

    style QLoRA_Path fill:#3fb950,color:#fff
    style Unsloth fill:#3fb950,color:#fff
    style DPO fill:#1f6feb,color:#fff
    style RAG fill:#8957e5,color:#fff
    style Prompts fill:#d29922,color:#fff
    style SFT fill:#1f6feb,color:#fff
    style Merge fill:#3fb950,color:#fff
```

— POCKET NOTES —

1. LoRA + QLoRA + DoRA + Adapters + Prefix Tuning

```
LoRA: add small trainable connections alongside frozen model. THE STANDARD.
  Trainable: 0.1-1% of parameters. Quality: 95-99% of full fine-tune.
  Rank r=16 default. Merge after training = zero overhead.
QLoRA: compress model to 4-bit THEN add LoRA. DEFAULT CHOICE.
  70B model: 140GB -> 35GB. Fits on 1 GPU!
DoRA: smarter LoRA (separate direction/strength). +2-5%. 2024.
Adapters: insert layers between blocks. Older. LoRA preferred.
Prefix Tuning: learn perfect instruction words. No weight changes.
Unsloth: 2x faster LoRA, 60% less memory. Always use.
LR: LoRA 2e-4 | QLoRA 1e-4 | Full 2e-5
Multiple adapters: one base model, swap per task (like phone apps)
```

2. When, Why, Hardware, Distributed Training

```
Full fine-tune: update ALL parameters. The expensive option.
Analogy: culinary school (2 years) vs cooking class (1 weekend)
Use when: new domain (medical/legal), max quality needed, small models (<1B)
PEFT better: task adaptation, budget limited, quality gap <2%
VRAM: 7B=28GB | 13B=52GB | 70B=280GB | 405B=1.6TB
QLoRA: divide VRAM by ~4. 70B=35GB on 1 GPU.
Distributed: FSDP (single machine) | DeepSpeed ZeRO-3 (multi-machine)
Always: bf16 + gradient checkpointing + save often
Cost: 7B full ~$16 | 70B full ~$640 | 70B QLoRA ~$8
Try QLoRA first. Full only if >2% quality gap.
```

3. Alpaca, ShareGPT, ChatML + Data Quality

```
3 Formats:
  Alpaca: {instruction, input, output}. Single-turn. Like homework.
  ShareGPT: {conversations:[...]}. Multi-turn chat.
  ChatML: <|im_start|>role\
content. Production standard.
Quality > Quantity: 1,000 curated = 52,000 auto-generated (LIMA paper)
Checklist: diverse, consistent, balanced, clean, human-verified 5%
Start: 1,000 examples. Add more only if eval improves.
Data poisoning: few bad examples ruin the whole model.
Dedup: remove near-identical examples.
Framework: Axolotl (supports all formats)
```

4. Reward Modeling, PPO, DPO, KTO

```
RLHF: train food critic (reward model) + RL (PPO). Complex. GPT-4/Claude method.
DPO: train directly from preference pairs. THE MODERN STANDARD.
  Simple, stable, fast. Needs [prompt, good, bad] data.
KTO: only needs thumbs up/down. Easiest data collection.
RLAIF: GPT-4 judges instead of humans. 80% quality, 10% cost.
Pipeline: SFT first (instruction following) -> THEN alignment
  Never skip SFT. Alignment on bad SFT = polishing a brick.
Data: 5K-10K preference pairs for DPO.
```

```
Library: trl (DPOTrainer, KTOTrainer). pip install trl.
Decision: pairs=DPO | thumbs=KTO | max quality=RLHF | no humans=RLAIF
```

5. Self-Instruct + Evol-Instruct + Quality Control

```
Self-Instruct: 175 seeds -> GPT-4 generates thousands -> filter for quality
Evol-Instruct: make questions progressively harder (level 1->2->3)
Quality control:
  LLM-as-judge: score 1-10, keep only 7+
  Dedup: remove near-identical examples
  Diversity: cover many topics, not just one
  Human spot-check: review 5% sample
  Generate 2x, keep top 50%
Cost: 10K via GPT-4 = $50-100 (vs $2K+ human writers)
Seed quality = output quality
Mix: 80% synthetic + 20% human = best balance
```

6. Domain Adaptation + When vs Fine-Tuning

```
Continued Pre-Training: feed raw domain text. Learns KNOWLEDGE.
Analogy: immersion learning (living in France vs taking a French class)
Pipeline: CPT (read textbooks) -> SFT (practice problems) -> DPO (get feedback)
Mix: 80% domain + 20% general (prevent forgetting)
LR: 1e-5 to 5e-5 (lower than SFT, gentle soaking)
Minimum corpus: 1GB. Less = not worth it.
Cost: 7B on 10GB text = ~$200-500
Always follow CPT with instruction tuning
Eval: domain perplexity should steadily decrease
```

7. Managed Platforms Compared

```
OpenAI: GPT-4o-mini/3.5/4o. Upload JSONL. ~$25/1M tokens. Simplest.
Together AI: Llama/Mistral. ~$5-10/1M tokens. Download the model.
Anthropic: enterprise only. Custom Claude.
Self-hosted: Axolotl + Unsloth. Full control. Data never leaves.
Decision: simplest=OpenAI | cheap+open=Together | full control=self-host
Managed cheaper: <1M tokens/day. Self-host cheaper: >1M tokens/day.
Managed required: proprietary models. Self-host required: data privacy.
Common error: data format mismatch. Match API format exactly.
Iterations: fine-tune->eval->add failure data->retune (3-5 times)
```

— INTERVIEW PREP —

1. LoRA + QLoRA + DoRA + Adapters + Prefix Tuning

```
SAY THIS: 'QLoRA is my default. It compresses the base model to 4-bit and applies LoRA adapters, letting me fine-tune a 70B model on a single GPU. After training, I merge adapters into the base for zero inference overhead. For slightly better quality, DoRA is a drop-in replacement.'

Q: How do you choose LoRA rank?
A: 'Start at r=16. If the task is complex, go to 32-64. I benchmark both on my eval set and pick the best quality-to-cost ratio. Higher rank = more trainable parameters = better capacity but slower training.'

Q: When would you NOT use PEFT?
A: 'When I need to change the model fundamentally -- like teaching it an entirely new language or domain from scratch. That needs full weight updates via continued pre-training. Also for very small models (<1B) where PEFT overhead is not worth it.'

Q: How do you serve multiple LoRA adapters?
A: 'One base model in memory, swap LoRA adapters per request. vLLM supports this natively. One 70B model + 50 LoRA adapters = 50 specialized models at the memory cost of 1.'
```

2. When, Why, Hardware, Distributed Training

SAY THIS: 'I default to QLoRA and only escalate to full fine-tuning when benchmarks show a meaningful quality gap -- typically greater than 2%. For a 70B model, QLoRA costs ~$8 vs ~$640 for full. I always try the cheaper option first and let the eval set decide.'

Q: When is full fine-tuning unavoidable?
A: 'Continued pre-training on domain corpora -- feeding the model 10GB of medical literature. PEFT adapters cannot absorb that volume of new knowledge. Also for very small models where PEFT overhead is disproportionate.'

Q: How do you handle multi-GPU training?
A: 'DeepSpeed ZeRO Stage 3 for maximum memory efficiency. Axolotl framework abstracts the complexity. Always enable gradient checkpointing and bf16 mixed precision.'

3. Alpaca, ShareGPT, ChatML + Data Quality

SAY THIS: 'I follow the LIMA principle -- data quality over quantity. I start with 1,000 carefully curated examples covering diverse tasks. I decontaminate against my eval set, verify a 5% sample, and balance task distribution. Only after confirming quality do I scale up.'

Q: How do you create instruction tuning data?
A: 'Three sources: (1) Domain experts write the best examples (expensive but highest quality). (2) GPT-4 generates synthetic examples with human review (good balance). (3) Converting existing data like logs into instruction format. I always mix sources and human-review a sample.'

Q: How many examples do you need?
A: 'Start with 1,000. Train, evaluate, check where the model fails. Generate more examples targeting failure cases. This targeted approach is 10x more efficient than generating 100K random examples.'

4. Reward Modeling, PPO, DPO, KTO

SAY THIS: 'Two-stage approach: SFT first for instruction following, then DPO for alignment. DPO is my default because it is simpler and more stable than RLHF with similar quality. For implicit feedback, KTO.'

Q: RLHF vs DPO -- when each?
A: 'DPO for 90% of cases. RLHF when you need an explicit reward model you can inspect and debug -- like safety-critical applications where you want to understand WHY the model prefers certain outputs.'

Q: How do you collect preference data?
A: '(1) Human annotators rank A vs B (expensive, highest quality). (2) RLAIF with GPT-4 as judge (80% quality, 10% cost). (3) Production feedback (thumbs up/down, clicks). I start with RLAIF, validate with human spot-checks.'

5. Self-Instruct + Evol-Instruct + Quality Control

SAY THIS: 'I use synthetic data generation with strict quality control. Self-Instruct for volume from curated seeds, Evol-Instruct for progressive difficulty. I generate 2x what I need, filter with LLM-as-judge, deduplicate, and human spot-check 5%.'

Q: Is synthetic data as good as human-written?
A: 'For instruction tuning: 80-90% as good when properly filtered. For safety and alignment: no -- you need real human judgments for edge cases. Best approach: synthetic for volume + human for critical examples.'

Q: How to prevent GPT-4 artifacts in synthetic data?
A: 'Post-process to remove verbal tics (Certainly!, As an AI). Vary generation temperature. Use multiple different prompts. Mix synthetic with human examples at 80/20 ratio.'

6. Domain Adaptation + When vs Fine-Tuning

SAY THIS: 'I use continued pre-training when the base model lacks deep domain knowledge. I always mix 80% domain and 20% general data to prevent catastrophic forgetting, and follow CPT with instruction tuning to teach the model to use its new knowledge.'

Q: CPT vs RAG for domain knowledge?
A: 'RAG retrieves facts at query time -- good for changing information. CPT bakes knowledge into weights -- good for deep understanding of terminology, reasoning patterns, conventions. Best: CPT for foundational knowledge + RAG for current facts.'

Q: How do you know if CPT worked?

A: 'Domain perplexity drops 20-50%. The model uses domain terminology naturally. Benchmark scores improve. I compare: same SFT with vs without CPT.'

7. Managed Platforms Compared

SAY THIS: 'For managed fine-tuning, I choose based on model access and control needs. OpenAI for simplest workflow. Together AI for open-source at lower cost. Self-hosted with Axolotl when I need full control or have privacy requirements.'

Q: OpenAI fine-tuning vs self-hosted?
A: 'OpenAI when: using GPT models, want zero GPU management, data is not sensitive. Self-hosted when: need open models, data privacy, full hyperparameter control, or cost optimization at scale. The crossover point is roughly 1 million tokens per day.'

Q: How do you know fine-tuning worked?
A: 'Held-out test set accuracy improves vs base model. Check for overfitting: training loss drops but eval loss increases. Production A/B test: 10% traffic to fine-tuned model, measure quality.'